TechIT Network - Platform Live-Execution Report (v3)

Full coverage: the approved plan (R, E, L, M, O, P, C, X), the original founder questions, the 1M-user scalability contract, the per-phase definition-of-done, what is left, the deployment recommendation, and answers to the four live deployment review questions.

Prepared: 2026-10-14. Repositories: ai-router, BACKEND, new-frontend.

v3 closes the three items v2 left open - E peer/cohort comparison, O org program/cohort execution scoping, and the remaining X mock fallbacks - and adds a one-line fix for the /refresh rate-limit lockout found during review.

1. Executive summary

The program makes one Execution Ledger the single source of truth, then adds thin lifecycle, consent and delivery layers on top. Eight phases were planned and all eight are now fully done (R, E, L, M, O, P, C, X):

- Phase E - execution intelligence now covers all three founder asks: vs. your own start, vs. similar startups, and cumulative improvement.

- Phase O - the organization dashboard reads execution intelligence scoped by the caller's organization, and a selected program/cohort, not the caller's own workspace.

- Phase X - no silent mock fallback remains in any frontend API client.

Everything below is committed and pushed. This v3 replaces v2 (which in turn replaced the first report).

2. Traceability: original founder questions -> status

- AI router allows only OpenAI + Anthropic keys -> Phase R -> DONE.
- All models (free or paid) must make real API calls -> Phase R -> DONE.
- Subscribers get free publicity of a completed product -> Phase P -> DONE.
- Investor signs consent to keep founder data private -> Phase C -> DONE.
- Execution intelligence: vs. start / vs. similar startups / cumulative -> Phase E -> DONE (baseline delta, anonymous industry+stage cohort percentile, cumulative totals).
- Institutions/incubators monitor startups in their program -> Phases O + M -> DONE (invites/onboarding/roster + per-program/cohort execution scoping on the org dashboard + platform metrics).
- Preserve evidence across idea -> collab -> roadmap -> prototype -> Phase E/L ledger -> DONE (append-only execution events + lifecycle derived from them).
- FastTrack codebase/prototype continues in the workspace -> Phase L -> DONE.
- Execution intelligence visualised, subscriber-gated -> Phase E -> DONE.
- Build -> launch transition tracked automatically -> Phase L -> DONE.
- FastTrack MVP vs already-launched detection -> Phase L -> DONE (auto-infer + explicit override).
- Org programs: invite/onboard/track; other org UI without real backend -> Phase O -> DONE for programs/invites/roster/scoping; remaining generic-CRUD org pages are a product decision (section 6), they are backed by live rows, not fake data.
- All AI models used via routing, not only OpenAI/Anthropic -> Phase R -> DONE.
- Track startups/MVPs/prototypes/milestones + surface on dashboards -> Phase M -> DONE.

Plus: everything live, real, functional (no mocks) -> Phase X -> DONE.

3. Phase detail

R - Router: all providers real and validated (DONE)

Registry declares 9 providers with real adapters, but credential health/readiness covered only OpenAI + Anthropic and the route chain appended unconfigured providers. Delivered: provider credential health generalised to every registry provider (base_url + headers), optional providers probed only when a key is present; readiness checks every configured provider (OpenAI/Anthropic stay mandatory); select_chain excludes unconfigured providers so a task never fires at a keyless model. Files: provider_key_health.py, routing_engine.py, .env.example, tests.

E - Execution Ledger + Intelligence depth (DONE)

Delivered: the canonical execution projection adds lifetime, first-week baseline, window-vs-baseline delta, a per-day sparkline and cumulative totals, all computed from the same persisted events (no second engine). v3 adds the plan's cohort block: an ANONYMOUS comparison of projects in the SAME industry AND stage, metric = completed milestones, withheld until the cohort has at least 5 peers (COHORT_MIN_SIZE = 5) so no individual peer is identifiable. It returns the founder's value, percentile and cohort median. Frontend: subscriber-gated visual progress (baseline vs now, sparkline, cumulative) plus "Ahead of N% of M similar industry/stage startups". Files: executionIntelligenceService.js (cohortFor), executionScopeGuard.test.js, ExecutionProgress.tsx, executionIntelligence.ts.

L - Lifecycle: build->launch + FastTrack honesty (DONE)

Delivered: lifecycleService reconciles stage from observable evidence (code events, files, docs, build path, MVP milestones, validations, verified deployments, traction), monotonic, with a manual override the reconciler always keeps. FastTrack gets a product-stage picker, an explicit "already launched?" and repo-based inference. project_id/workspace_id persist and the analysis continues in the workspace. Files: lifecycleService.js, stage_inference.py, FastTrackPanel.tsx, tests.

M - Platform metrics (DONE)

Delivered: one aggregate from the ledger (startups, prototypes, MVP completed, launches, growth, milestones, per-founder breakdown), scoped by role; platform-wide is admin-only. Surfaced on founder, org and admin dashboards. Files: platformMetricsService.js, PlatformMetricsStrip.tsx, ImpactReporting.tsx.

O - Org programs: invites, onboarding, tracking, scoping (DONE)

Delivered: program<->founder link with invite tokens and acceptance (programEnrollments), an auto-populated scoped roster, the Incubator program manager, and a founder invite-acceptance page. v3 wires the dashboard: the org Dashboard and the program manager render ExecutionIntelligencePanel with organizationId and, for a selected program, programId + cohortId, so the panel reads execution intelligence scoped to that organization/program/cohort instead of the caller's own workspace. The backend scope guard already enforces active membership of the named organization and pins program/cohort ids to that same tenant, so an organization can never read another tenant's cohort. Files: domainService.js (programEnrollments), Incubator.tsx, org/Dashboard.tsx, ExecutionIntelligencePanel.tsx, ProgramInviteAccept.tsx.

P - Publicity for subscribers (DONE)

Delivered: a project reaching launched for a subscribed user generates a TechIT Moment + feed post; a publicity notification-preference category gates delivery (opt-out); cost is 0 for subscribers. No new broadcast system. Files: publicityService.js, notifications.ts, Settings.tsx, NotificationsPanel.tsx.

C - Consent (DONE)

Delivered: a canonical consent scope catalog (approved_evidence_metadata shareable; execution_metrics and cohort_benchmarks SENSITIVE). Request/decision carry grantedScopes; approving without naming scopes grants ONLY the non-sensitive requested scopes; rejection grants nothing; unknown scopes dropped (fail-closed). Ownership-scoped execution boundary: organization/program/cohort scopes require an active membership of that org (organizationId pinned so a program/cohort id cannot point at another tenant); hackathonId requires ownership; admins may scope any tenant. Investors are stripped of ALL tenant scopes on that route, so an investor can NEVER read an organization cohort that is not theirs; an investor may only scope a startup they are authorized for and only with the granted execution scope. Investors keep intelligence on the mentorship programs THEY run (mentorId == investorId): the room's project and each active mentee's project. Investor execution velocity and code-commit evidence stay private until execution_metrics is granted. AI-router registers the new scopes fail-closed. Files: consentScopes.js, investorTrustAccessService.js, executionIntelligenceService.js, investorIntelligenceService.js, domainController.js, trust_investor_read_model.py, TrustCenter.tsx, InvestorTrustDashboard.tsx, tests.

Consent flow, as shipped (see also section 7, question 5): the investor opens a startup in the investor dashboard and, when not yet approved, clicks "Request additional Trust access"

(requestInvestorTrustAccess sends the full catalog as requestedScopes). The FOUNDER decides in Trust Center: per-request checkboxes, sensitive scopes unchecked by default, "Approve selected" calls decideFounderTrustAccessRequest with the chosen grantedScopes. The investor read then sends only the founder's grantedScopes; the investor can never self-grant.

X - Everything live (DONE)

Delivered: removed the spurious MCP_ENABLED gate on execution intelligence (it reads local stores, not the connector surface; MCP_ENABLED now gates only /api/mcp and the GitHub->vault bridge); FastTrack/incubation client surfaces real pipeline errors instead of a silent null; and v3 removes the remaining silent withFallback mock fallbacks from all six clients (alerts, audio, gsis, training, workspace, tourGuide). A grep for withFallback across frontend/src/lib/api now returns none outside the client/config plumbing. Files: alerts.ts, audio.ts, gsis.ts, training.ts, workspace.ts, tourGuide.ts.

4. 1M-user scalability contract

- Row-level indexed reads/writes only; pagination + bounded candidate limits ->
all new reads are owner/project/scope-bounded (consent lookup is per project+investor; cohort is a single indexed industry+stage query that only returns an anonymous aggregate; metrics aggregate runs over scoped rows).

- Respect config/scalability_slos.json (read p95 <= 500ms, 0.999 availability) and
scripts/capacity_model.py scenarios up to 1,000,000 users.

- Aggregates precomputed/rolled-up, never full scans per request -> Phase M reads
persisted project/milestone rows and rolls up; the cohort query is bounded to one industry+stage.

- Migrations additive, reversible; no destructive schema changes -> none shipped.
Pre-deploy step: run the capacity model against the target profile and confirm the SLOs with the row-level stores authoritative.

5. Verification and definition-of-done

Tests (all passing). Supertest suites were run outside the sandbox (they bind a local port).

- BACKEND: full suite 970/970 (127 files), incl. the new cohort tests in
executionScopeGuard.test.js and the consent-seeded code-execution projection test.

- new-frontend: full suite 271/271 (64 files); tsc -b clean; eslint on touched
files clean.

- ai-router: 284 passed, 1 skipped.
Definition of done (per phase): implementation end-to-end; tests added and passing; no regressions; docs updated; production-readiness and scalability maintained; no mock/fake path on the surface touched.

- R/E/L/M/P/C/X: met.
- E: met for self-comparison AND peer/cohort comparison.
- O: met for invites/onboarding/roster AND per-program/cohort execution scoping.
- X: met - no mock fallback remains in any frontend API client.

6. What is left (open items, in priority order)

- Phase O - decide which organization pages (AI Ops, Market-Readiness,
Community/Hangout, Integrations, Marketplace, Talent Pool, Settings) need bespoke backend automation versus staying generic bound collections. These are live, real rows today; this is a product-scope decision, not a mock.

- Deployment prerequisites (environment, not code - see section 7).
- Separate: repository Dependabot backlog (BACKEND 47 advisories incl. 13 critical;
ai-router 1 critical; new-frontend 1 low). Triage outside this program.

7. Deployment recommendation

Environment (cannot be flipped from the repo):

- Row-level stores must be authoritative in production. .env.example still ships sqlite DEV defaults; production must set DOMAIN_SOURCE=postgres and the sibling *_READ_SOURCE / *_WRITE_SOURCE flags, otherwise the platform runs on the sqlite snapshot fallback.

- messaging-backend already ships a pgx/Postgres store (internal/store/postgres); the deployed instance must point it at the platform database (techit_msg).

- MCP_ENABLED=true only where connectors are actually deployed; it no longer affects core read models.

Rollout:

- Merge order: backend first (it owns the consent authority and the read gate), then ai-router (read-model scope names), then new-frontend (UI). The backend is backward compatible: an older frontend never asks for sensitive scopes.

- Gate the release on the production config check passing with every *_SOURCE flag set to postgres; do not promote with the sqlite fallback active.

- Roll the consent change out dark: existing approvals keep working (they had no grantedScopes, so they resolve to the baseline scope). No data backfill needed.

- After deploy verify four live behaviours: (a) an investor request for another org's cohort returns 403; (b) execution metrics are hidden until the founder grants execution_metrics; (c) /ready on the router reflects every configured provider, not just OpenAI/Anthropic; (d) /refresh no longer consumes the /signin rate-limit budget.

- Rollback: revert the per-repo commits; no schema or data rollback is needed because nothing destructive shipped.

7.1 Answers to the four review questions

Q1. Is JWT_ALGORITHM=RS256 intended for Node? Messaging has no JWT_PUBLIC_KEY and is still the Sept 23 build, so it cannot verify those tokens. Publish a messaging image / merge a CI build?

- Yes, RS256 is intended: jwtKeyService.js forces RS256 in production/staging (HS256 is refused there), and .env.example ships RS256 PEM keys.

- The Go messaging service already supports RS256 verification with JWT_PUBLIC_KEY (internal/auth/jwt.go picks RS256 when the key is present and refuses HS256 in production; internal/config/config.go requires JWT_PUBLIC_KEY outside dev/test). That support landed in commit b9ca9c1 (2026-09-30). The Sept-23 image predates it, which is exactly why it rejects platform tokens today.

- Fix: rebuild/publish the messaging image with JWT_PUBLIC_KEY set. Use .github/workflows/deploy-messaging-ec2.yml - it is gated on a green "TECHIT Backend Services" run and can also be run via workflow_dispatch; it provisions JWT_PUBLIC_KEY, the RDS URL and techit_msg, and rebuilds on the box. Because building on the 909MB t3.micro froze it, prefer workflow_dispatch from CI (or build the image elsewhere and pull), not a manual on-box build.

Q2. REQUIRE_AI_EXECUTION_GRANT=true on ai-router: what path supplies the grant, and does the frontend send it?

- The grant is backend-minted, never frontend-minted. BACKEND issueExecutionGrant (usageGrantService.js) signs a short-lived token and is exposed at POST /usage/grant (routes/usageSettlement.js, requireUsageGrantIssuer).

- ai-router reads it from the X-AI-Execution-Grant header in get_user_context (main.py) and, when the flag is on, rejects any request without it.

- The frontend does NOT send the header. Only BACKEND does, and today only on the trust-verification path (aiRouterClient.analyzeVerificationEvidence -> /api/v1/incubation/evidence/research). Every other router call from aiRouterClient.js (gsis/compute, workspace/execute, workspace/conversation, investor advisory/evi, organization advisory, support, training) omits the header. So flipping REQUIRE_AI_EXECUTION_GRANT=true without extending those callers to mint and attach a grant will 403 them.

- Also align the algorithm: BACKEND signs HS256 by default

(AI_EXECUTION_GRANT_ALGORITHM | | 'HS256') while the ai-router verifier defaults to RS256 in production/staging. Either set AI_EXECUTION_GRANT_ALGORITHM=RS256 on both sides with AI_EXECUTION_GRANT_SECRET as the RS256 key, or keep both on the same shared HMAC secret. Do not leave the defaults mismatched in production.

- Recommendation: only enable the flag once every router caller mints a grant (a shared helper), or keep it false and rely on the backend as the single entry point. Do not enable it as-is.

Q3. ai-router goes through pgbouncer on 6432 with db "techit_db", but its migration expects techit_ai_db. Which is intended?

- techit_db is intended. There is no techit_ai_db anywhere in the repos or their history. ai-router uses DATABASE_URL (default .../techit_db in .env.example and docker-compose.yml) and falls back to AI_ROUTER_IDENTITY_DATABASE_URL (.../techit_core) only for the identity/session store. The messaging service uses its own techit_msg; the platform/backend owns the rest of techit_db.

- Action: point the box's ai-router DATABASE_URL at techit_db (via the 6432 pgbouncer proxy). If a separate AI database is genuinely wanted, that is a deliberate change across env + migrations, not an existing expectation - and it is not needed for correctness.

Q4. Rate limiter: /signin and /refresh share signinLimit (5 per minute), so the frontend's /refresh retries lock sign-in out.

- Confirmed and fixed in this commit. routes/auth.js now gives /refresh its own refreshLimit (30/min), leaving signinLimit (5/min) for credential entry (/signin, /forgot-password, /reset-password). A refresh burst can no longer exhaust the sign-in budget. BACKEND auth suite 48/48; full suite 970/970.

Q5 (review follow-up). Should an investor need consent to see that a startup ships, or only for financial/sensitive execution data?

- Today the code gates BOTH execution velocity and verified code-commit count behind the sensitive execution_metrics scope (investorIntelligenceService.js), so an investor with only approved_evidence_metadata sees milestones/mentorship/last activity but not commits or velocity.

- Financial data is ALREADY on its own consent scope: the ai-router investor read model gates its revenue source behind revenue_metrics (trust_investor_read_model.py), separate from execution_metrics and cohort_benchmarks.

- Recommendation: the intended split is sensible - baseline shares a non-financial "are they shipping?" signal (verified commits, milestone completion, activity recency), while velocity/EVI and financials (revenue, burn, runway, cap table, fundraising) require execution_metrics / revenue_metrics. That requires (a) moving verified-commit evidence out of the execution_metrics gate, and (b) adding revenue_metrics to the BACKEND request catalog so it is requestable and shows as sensitive in Trust Center. This is a small, deliberate change; it is flagged here rather than shipped silently because it widens what an investor sees by default. Confirm and it will be implemented next.

8. Commit inventory (this program)

BACKEND (main): 52b15de E-cohort + tests + refresh-limit, e92ba69 X, 67b15a0 C, 1945b5b P, 1748880 O, bfaaf09 metrics-scope, c860244 M, 069f56a L, 98f779b E, 44da07f plan, e09ed9f docs. ai-router (fix/provider-key-readiness): 291f29e R, 886909d L, 6d4f9db C, b87338b docs, 1db1a73 plan. new-frontend (docs/frontend-debug-access): e48d7f6 E/O/X, 5292936 X, c809dd9 C, 36d2f03 P, 5235e29 O, 40ef646/4b3334a M, ba2e51c L, ed41aeb E, 34d70ef docs, c8767e4 plan.